# CampusCore – Student Registration and Management System using Java Servlets, JSP, JDBC and MySQL

**A PROJECT REPORT**

*Submitted by*

**Bhumi Jindal (26MCD10071)**

*Submitted to*

**Ms. Tanisha**

*in partial fulfillment for the award of the degree of*

## Master of Computer Applications

**IN**

Data Science

**2026-28**

**Chandigarh University**

2026-28

# BONAFIDE CERTIFICATE

Certified that this project report **"CampusCore – Student Registration and Management System using Java Servlets, JSP, JDBC and MySQL"** is the bonafide work of **"Bhumi Jindal (26MCD10071)"** who carried out the project work under my/our supervision. The content and findings presented herein are authentic and reflect the original work conducted during the course of this project.

| | |
|---|---|
| **SIGNATURE** | **SIGNATURE** |
| Name: Bhumi Jindal | Name: Ms. Tanisha |
| UID: 26MCD10071 | Designation: Asst. Professor |
| Department: MCD - 2/A | SUBJECT CODE: E20679 |

Submitted for the project viva-voce examination held on ____ / ____ / _____

# ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all the individuals who have contributed to the success of this project. Developing a complete database-driven web application using Java Servlets, JSP, JDBC and MySQL presented several challenges, and I am immensely grateful for the support and guidance I received along the way.

I extend my sincere appreciation to my project supervisor, Ms. Tanisha, for the invaluable support and encouragement throughout the duration of this project. I also owe a special debt of gratitude for the guidance, expertise and motivation that were instrumental in overcoming obstacles and achieving the goals of this work.

I would also like to thank all the faculty members of the University Institute of Computing for their valuable suggestions and feedback, which helped me improve the project and explore new features.

A heartfelt thank you goes to my parents for their unwavering support and encouragement. Their belief in me inspired me to pursue this project with determination. I am grateful to all those who, directly or indirectly, contributed to this project.

**Bhumi Jindal**
**UID: 26MCD10071**

# TABLE OF CONTENTS


| Contents | Page No. |
|---|---|
| **Abstract** | **iv** |
| **CHAPTER 1: INTRODUCTION** | **1** |
| 1.1. Project Scope and Project Planning | 1 |
| 1.2. Client Identification and Recognition of need | 1 |
| 1.3. Recognition and Knowledge of relevant contemporary issues | 1 |
| 1.4. Task Identification | 2 |
| 1.5. Technologies Used | 2 |
| 1.6. Timeline of the project | 2 |
| **CHAPTER 2: LITERATURE REVIEW/BACKGROUND STUDY** | **3** |
| 2.1 Timeline of the reported problem | 3 |
| 2.2 Existing solutions | 3 |
| 2.3 Bibliometric analysis | 3 |
| 2.4 Review Summary | 3 |
| 2.5 Problem Definition | 3 |
| 2.6 Goals/ Objectives | 3 |
| **CHAPTER 3: DESIGN FLOW/PROCESS** | **4** |
| 3.1 Evaluation & Selection of Features | 4 |
| 3.2 Design Constraints | 4 |
| 3.3 Analysis of Features and finalization subject to constraints | 4 |
| 3.4 Design Flow | 5 |
| 3.5 Design Selection | 5 |
| 3.6 Implementation plan/methodology | 5 |
| **CHAPTER 4: RESULTS ANALYSIS AND VALIDATION** | **6** |
| 4.1 Implementation of the solution | 6 |
| **CHAPTER 5: CONCLUSION AND FUTURE WORK** | **8** |
| 5.1 Conclusion | 8 |
| 5.2 Future Scope | 8 |
| **APPLICATION CODE OF PROJECT** | **9** |
| **REFERENCES** | **11** |
| **APPENDIX** | **11** |
| **USER MANUAL** | **11** |
| **GITHUB REPOSITORY** | **12** |


## List of Figures


| List of Figures | Page No. |
|---|---|
| Figure 1: Development lifecycle of the project | 2 |
| Figure 2: Architecture of CampusCore | 5 |
| Figure 3: Student Registration page | 6 |
| Figure 4: Duplicate Student ID validation | 6 |
| Figure 5: Registered Students list | 6 |
| Figure 6: Search Student | 6 |
| Figure 7: Edit Student | 7 |
| Figure 8: Delete Student | 7 |
| Figure 9: Database records in MySQL | 7 |
| Figure 10: GitHub repository of the project | 12 |

# ABSTRACT

Maintaining student records manually in registers or spreadsheets is slow, error-prone and difficult to search or update. This project, **CampusCore**, is a Java-based web application that provides a simple and reliable way to register and manage student records through a browser. It is built on the standard Java web stack: JavaServer Pages (JSP) for the user interface, Jakarta Servlets as controllers, JDBC for database access, and MySQL for persistent storage, deployed on Apache Tomcat and built with Maven.

The system implements the complete CRUD cycle: a student can be **registered** with a Student ID, name, email, department and year; **viewed** in a tabular list; **searched** by Student ID or name; **edited**, while the Student ID remains fixed; and **deleted**. Before a new record is stored, the application checks whether the Student ID already exists and, if so, rejects the entry with a clear message, preventing duplicate registrations. All database operations use parameterised `PreparedStatement` queries, which protect the system against SQL injection.

The application was designed with a layered structure (model, servlet, utility and view layers), tested on a local Tomcat–MySQL set-up, and published on GitHub. This report describes the scope, background, design, implementation, results and future scope of the project, and includes screenshots of the working system.

**Keywords:** Student Management, Java Servlets, JSP, JDBC, MySQL, CRUD, Apache Tomcat, Maven.

## List of Abbreviations

| Abbreviation | Definition |
|---|---|
| JSP | JavaServer Pages |
| JDBC | Java Database Connectivity |
| CRUD | Create, Read, Update, Delete |
| SQL | Structured Query Language |
| DBMS | Database Management System |
| MVC | Model–View–Controller |
| POJO | Plain Old Java Object |
| HTTP | HyperText Transfer Protocol |
| WAR | Web Application Archive |
| IDE | Integrated Development Environment |
| UI | User Interface |
| ER | Entity Relationship |

# CHAPTER 1.
# INTRODUCTION

Educational institutions handle a continuous flow of student data: admissions, registrations, contact details, departments and academic years. When this information is kept in paper registers or scattered spreadsheets, it becomes hard to keep consistent, slow to search, and easy to duplicate or lose. A small, database-backed web application can remove most of these problems at a low cost.

CampusCore is such an application. It lets an authorised user register students, view all records, search for a particular student, update details and delete records, all through a web browser. The project is deliberately built on the core Java web technologies (Servlets, JSP and JDBC) rather than a large framework, so that every layer of a web application – request handling, business logic, data access and presentation – is visible and well understood. It therefore also serves as a practical demonstration of how a classic three-tier, database-driven Java application is built.

## 1.1. Project Scope and Project Planning:

The scope of the project is the design and development of a web-based student registration and management system with a MySQL back end. The system covers: (i) registration of a student with five attributes (Student ID, name, email, department and year); (ii) automatic rejection of a duplicate Student ID; (iii) display of all registered students; (iv) search by Student ID or by (partial) name; (v) modification of an existing record, with the Student ID kept unchanged; and (vi) deletion of a record. Features such as login, attendance, marks and reporting are outside the present scope and are listed under future work.

**Project planning involved the following key phases:**

1. **Requirement analysis:** identifying the data to be stored and the operations required by an administrator.
2. **Database design:** designing the `students` table with `student_id` as the primary key and writing the SQL setup script.
3. **Back-end development:** implementing the model class, the JDBC connection utility and one servlet for each operation.
4. **Front-end development:** building the JSP pages (registration, list, search and edit) with a clean, consistent CSS layout.
5. **Testing and validation:** testing each CRUD operation and the duplicate-ID check against a local MySQL database.
6. **Documentation and publication:** preparing the README, screenshots and this report, and uploading the project to GitHub.

## 1.2. Client Identification and Recognition of need: -

The primary clients of the system are the administrative staff of a college or department who register and maintain student records. Secondary stakeholders are faculty members, who need quick access to student details, and students, who benefit from accurate records. Discussions of the typical pain points of manual record keeping gave the following needs: a single reliable store for student data; protection against registering the same student twice; quick look-up by ID or name; easy correction of mistakes; and a simple interface requiring no technical training. These needs were converted into the functional objectives listed in Section 2.6.

## 1.3. Recognition and Knowledge of relevant contemporary issues: -

Digitisation of academic administration is a major contemporary theme in education. Institutions are expected to keep accurate, retrievable records and to protect personal data such as names and email addresses. Common problems in small systems include duplicate or inconsistent records, weak input handling, hard-coded database credentials, and vulnerabilities such as SQL injection and cross-site

scripting. CampusCore addresses part of this set by using prepared statements for every query, enforcing a primary-key constraint with an explicit duplicate check, and keeping database access in a single utility class. The remaining issues – authentication, role-based access, output encoding and secure configuration – are acknowledged as limitations and are covered in Section 5.2.

## 1.4. Task Identification: -

- **Database task:** create `student_db` and the `students` table (`student_id`, `name`, `email`, `department`, `year`).
- **Model task:** create the `Student` class (a POJO) with a constructor, getters and setters.
- **Connectivity task:** write `DBConnection` to load the MySQL driver and return a JDBC connection.
- **Controller tasks:** write servlets for `/register`, `/show_all`, `/search`, `/edit` and `/delete`.
- **View tasks:** create `index.jsp`, `students.jsp`, `search.jsp` and `edit.jsp`.
- **Quality tasks:** test each operation, capture screenshots, document the work and push the code to GitHub.

## 1.5. Technologies Used: -

| Technology | Purpose in the project |
|---|---|
| Java (JDK 21) | Core programming language for the model, servlets and utilities. |
| Jakarta Servlet 5.0 | Controller layer; handles HTTP requests and responses. |
| JSP, HTML, CSS | View layer; renders forms and tables with a consistent style. |
| JDBC (MySQL Connector/J 9.3.0) | Connects the Java application to MySQL using prepared statements. |
| MySQL 8.0 | Relational database for persistent student records. |
| Apache Tomcat | Servlet container that runs the packaged web application (WAR). |
| Maven | Dependency management and WAR packaging (`pom.xml`). |
| NetBeans IDE, Git, GitHub | Development environment, version control and project hosting. |

## 1.6. Timeline of the project

The project was carried out in six phases, shown in Figure 1: requirement analysis, database design, back-end development, front-end development, testing and validation, and documentation with GitHub publication. The work is estimated at roughly six weeks, with the back end and front end progressing in parallel once the database design was fixed.

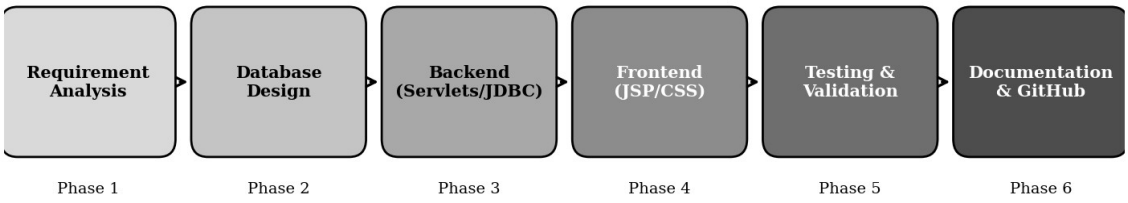



**Figure 1: Development lifecycle of the project**

# CHAPTER 2.
# LITERATURE REVIEW/BACKGROUND STUDY

## 2.1 Timeline of the reported problem:

Record keeping in education has moved from paper registers (before the 1990s) to desktop spreadsheets and standalone database programs, and then to web-based student information systems. Early web systems used CGI scripts; Java Servlets and JSP (late 1990s) offered a more structured, platform-independent alternative, later complemented by frameworks such as Spring MVC and Spring Boot. In every stage the reported problems have been the same: duplicated entries, inconsistent data, slow retrieval and the effort of manual updates.

## 2.2 Existing solutions:

**Spreadsheets** (e.g. Excel) are easy to start with but have no enforced uniqueness, no concurrent multi-user access and poor searchability at scale. **Commercial ERP systems** are feature-rich but expensive and excessive for a small department. **Framework-based applications** (e.g. Spring Boot with JPA) are powerful but hide the underlying mechanics. **Plain Servlet/JSP with JDBC** is lightweight, transparent and runs on any Tomcat server, which is the approach chosen here.

## 2.3 Bibliometric analysis:

A review of the textbooks and official documentation listed in the References shows the following.

**Key features**

- Separation of concerns (MVC): servlets act as controllers, JSP pages as views and plain Java classes as models.

**Effectiveness**

- Servlet/JSP applications are portable, stable, widely supported and easy to deploy on Tomcat.

**Drawbacks**

- Basic implementations often lack authentication, output encoding, pagination and automated tests.

## 2.4 Review Summary:

The literature agrees that a layered architecture, parameterised queries and constraint-based data integrity are the foundation of a dependable record-management application. Small projects often omit security features; CampusCore adopts the recommended foundations and lists the omissions as future work.

## 2.5 Problem Definition:

To design and develop a web-based system that allows an institution to **register, view, search, update and delete student records** in a MySQL database, while **preventing duplicate registration** of the same Student ID and providing a clear, easy-to-use interface that works on a standard Tomcat server.

## 2.6 Goals/ Objectives:

- To develop a web-based student registration and management system.
- To store student information persistently in a MySQL database through JDBC.
- To implement all four CRUD operations using Java Servlets and JSP.
- To prevent duplicate student registration by validating the Student ID.
- To provide search by Student ID or name, and facilities to update and delete records.

# CHAPTER 3.
# DESIGN FLOW/PROCESS

The design follows a standard three-tier approach. The presentation tier consists of JSP pages; the logic tier consists of servlets and a model class; and the data tier is a MySQL database accessed through JDBC. The design process moved from identifying the required features, through constraints and selection, to the final architecture and implementation plan described below.

## 3.1 Evaluation & Selection of Features: -

Candidate features were evaluated for their importance to a student-record system and for their effort. Features marked “Included” form the implemented system; the others are deferred.

| Feature | Reason | Priority | Decision |
|---|---|---|---|
| Student registration | Core requirement; creates records | High | Included |
| Duplicate ID validation | Maintains data integrity | High | Included |
| View all students | Basic read operation | High | Included |
| Search by ID / name | Fast retrieval of one record | Medium | Included |
| Edit student details | Corrects mistakes | Medium | Included |
| Delete student | Removes obsolete records | Medium | Included |
| Login / role-based access | Security; larger effort | Medium | Future |
| Attendance, marks, export | Extended modules | Low | Future |

## 3.2 Design Constraints: -

- **Technical:** the application must run on Java 21, Jakarta Servlet 5.0 (Tomcat 10 or later) and MySQL 8.
- **Data:** the Student ID is numeric and must be unique; name, email, department and year are mandatory.
- **Security:** all SQL must be parameterised; database credentials must not be published.
- **Resource and time:** a single-developer academic project, so the design avoids heavy frameworks.
- **Usability:** the interface must be simple enough to be used without training.

## 3.3 Analysis of Features and finalization subject to constraints:

Considering the constraints, all five data operations were finalised because they depend only on one table and need no extra infrastructure. Login and role-based access were deferred because they require a user table, password hashing and session management, which exceed the time available. A relational table was preferred to file storage because its primary key also guards against duplicate IDs.

## 3.4 Design Flow:

Figure 2 shows the flow of a request: the browser sends an HTTP request, a servlet processes it using a `PreparedStatement` on a JDBC connection from `DBConnection`, places the resulting `Student` objects in the request and forwards to a JSP page that renders the response.

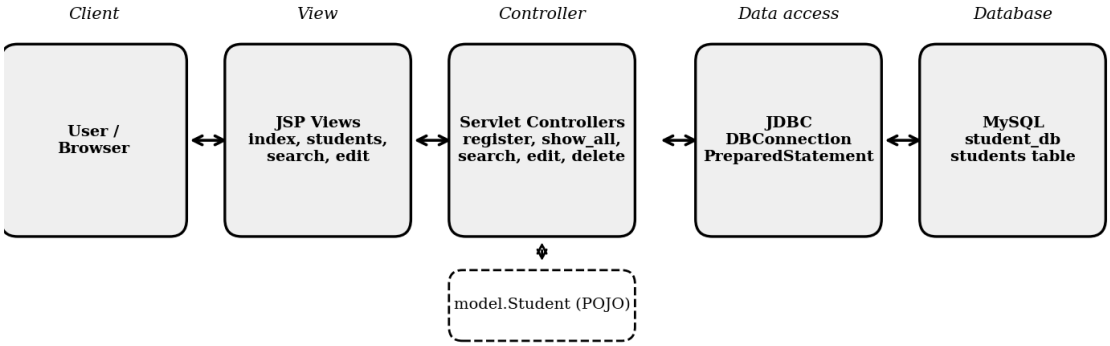



**Figure 2: Architecture of CampusCore**

The database consists of a single table, `students`:

| Column | Type | Constraint / description |
|---|---|---|
| student_id | INT | PRIMARY KEY – unique Student ID |
| name | VARCHAR(100) | NOT NULL – student name |
| email | VARCHAR(100) | NOT NULL – email address |
| department | VARCHAR(50) | NOT NULL – department / course |
| year | INT | NOT NULL – academic year |

## 3.5 Design Selection: -

A layered, MVC-style design was selected: the `model` package holds the `Student` POJO, the `servlet` package holds one controller per operation, the `util` package holds the connection helper, and the JSP files form the view. Compared with Spring Boot it has far fewer dependencies. Each servlet opens its connection with try-with-resources so that connections, statements and result sets are always closed.

## 3.6 Implementation plan/methodology

Development was incremental: database first, then the model and connection class, then each servlet with its JSP page, testing each operation before moving on. Table 3.1 lists the servlet mappings.

**Table 3.1: Servlet mappings**

| URL | Method | Servlet | Function |
|---|---|---|---|
| /register | POST | RegisterStudentServlet | Checks for an existing ID, then inserts the record |
| /show_all | GET | ShowStudentsServlet | Fetches all students ordered by ID and forwards to students.jsp |
| /search | GET | SearchStudentServlet | Finds records by exact ID or name (LIKE) and forwards to search.jsp |
| /edit | GET / POST | EditStudentServlet | GET loads a record into edit.jsp; POST updates it |
| /delete | GET | DeleteStudentServlet | Deletes the record with the given ID |

# CHAPTER 4.
# RESULTS ANALYSIS AND VALIDATION

## 4.1 Implementation of the solution:

The application was deployed on Apache Tomcat and tested against a local MySQL 8.0 database. The screenshots below show each module working.

**Registration and duplicate validation.** The registration page (Figure 3) accepts Student ID, name, email, department and year. On submission the servlet first queries the table for the ID; when the ID already exists, it stops and shows the “Student Already Registered!” message (Figure 4). Otherwise the record is inserted and the user is redirected to the student list.

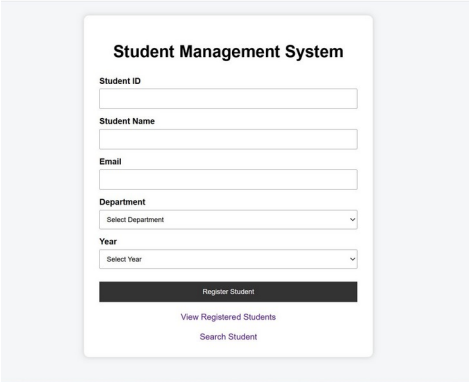



**Figure 3: Student Registration page**

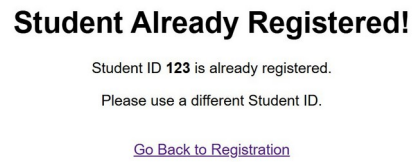



**Figure 4: Duplicate Student ID validation**

**View and search.** All registered students appear in a table ordered by Student ID (Figure 5) with Edit and Delete actions. The search page (Figure 6) returns records whose ID equals the entered number or whose name contains the entered text.

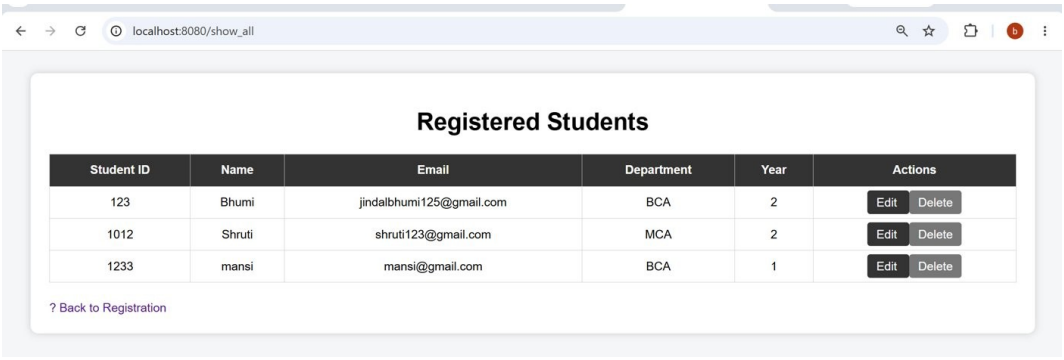



**Figure 5: Registered Students list**

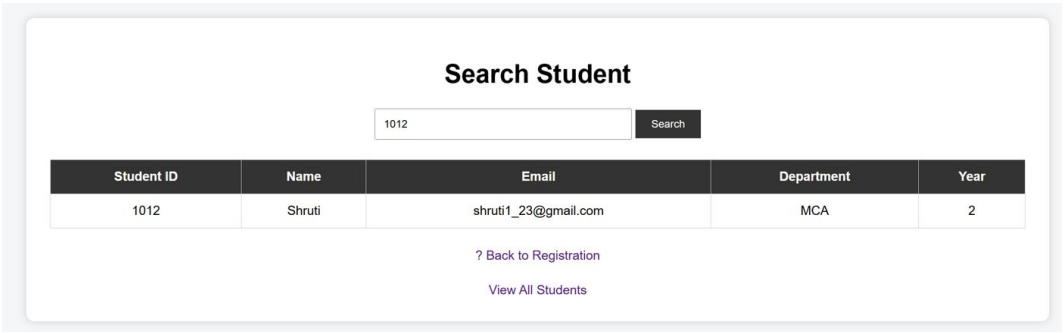



**Figure 6: Search Student**

**Edit and delete.** The edit form (Figure 7) is pre-filled with the stored values and keeps the Student ID unchanged; saving executes an `UPDATE`. Deleting (Figure 8) removes the record and refreshes the list.

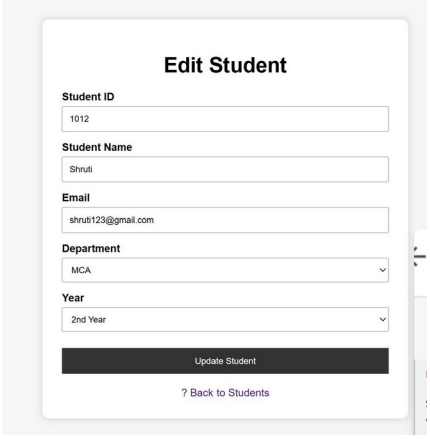



**Figure 7: Edit Student**

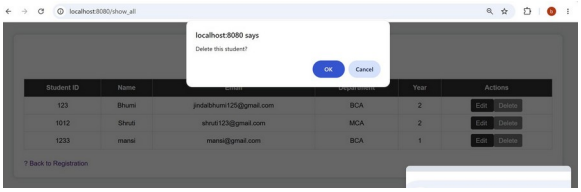



**Figure 8: Delete Student**

**Database verification.** Figure 9 confirms that the records created through the web interface are stored in the `students` table of `student_db`.

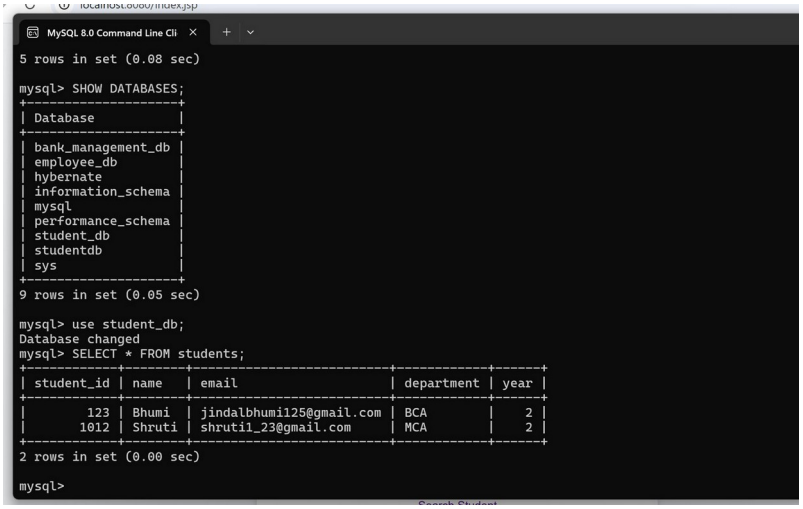



**Figure 9: Database records in MySQL**

## Test summary

| No. | Test case | Expected result | Result |
|---|---|---|---|
| 1 | Register a new student | Record stored; list is shown | Pass |
| 2 | Register an existing Student ID | “Already Registered” message; no insert | Pass |
| 3 | Open the student list | All records displayed in order of ID | Pass |
| 4 | Search by ID and by name | Matching records displayed | Pass |
| 5 | Edit name / email / department / year | Record updated; ID unchanged | Pass |
| 6 | Delete a student | Record removed from table | Pass |

# CHAPTER 5.
# CONCLUSION AND FUTURE WORK

## 5.1 Conclusion: -

CampusCore achieves its objectives: it provides a working, database-backed web application in which student records can be registered, viewed, searched, edited and deleted, and in which duplicate Student IDs are rejected. The layered design, the use of prepared statements and the try-with-resources pattern make the code clean, secure against SQL injection and free of resource leaks. The project also gave hands-on experience of the full life cycle of a Java web application – from database design and servlet development to deployment on Tomcat, testing, documentation and publication on GitHub.

The present version is a foundation rather than a finished product. It has no authentication, deletion is triggered by an HTTP GET link, user input is not yet validated on the server beyond number parsing, and the database password is kept in source code. These limitations are known and form the basis of the future scope.

## 5.2 Future Scope: -

- User authentication, login sessions and role-based access control (admin / staff).
- Moving database credentials to environment variables or a configuration file, and adding a connection pool.
- Server-side validation of all fields (e.g. email format), HTML output encoding, and deletion through a POST request with confirmation.
- Pagination and sorting for large lists; export of records to CSV/PDF.
- Additional modules: attendance, marks and results, student profiles and an admin dashboard.
- Automated unit and integration tests, and cloud deployment of the application.

# APPLICATION CODE OF PROJECT

The main source files are given below (package and import statements are omitted for brevity; the registration form in index.jsp posts to /register). The complete source is available in the GitHub repository. The database password is shown as a placeholder.

### 1. database/student_db.sql

```
CREATE DATABASE IF NOT EXISTS student_db;
USE student_db;

CREATE TABLE IF NOT EXISTS students (
  student_id INT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(100) NOT NULL,
  department VARCHAR(50)  NOT NULL,
  year       INT NOT NULL
);
```

### 2. util/DBConnection.java

```
public class DBConnection {
  private static final String URL =
    "jdbc:mysql://localhost:3306/student_db";
  private static final String USER = "root";
  private static final String PASSWORD =
    "YOUR_MYSQL_PASSWORD";

  public static Connection getConnection()
      throws Exception {
    Class.forName("com.mysql.cj.jdbc.Driver");
    return DriverManager.getConnection(
      URL, USER, PASSWORD);
  }
}
```

### 3. model/Student.java

```
public class Student {
  private int studentId;
  private String name, email, department;
  private int year;

  public Student() { }
  public Student(int studentId, String name,
      String email, String department,
      int year) {
    this.studentId = studentId;
    this.name = name;
    this.email = email;
    this.department = department;
    this.year = year;
  }
  // getters and setters for all fields
  public int getStudentId() {
    return studentId;
  }
  public void setStudentId(int studentId) {
    this.studentId = studentId;
  }
  // getName/setName, getEmail/setEmail,
  // getDepartment/setDepartment,
  // getYear/setYear ...
}
```

### 4. servlet/RegisterStudentServlet.java

```
@WebServlet("/register")
public class RegisterStudentServlet
    extends HttpServlet {
  protected void doPost(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    int studentId = Integer.parseInt(
      request.getParameter("studentId"));
    String name = request.getParameter("name");
    String email = request.getParameter("email");
    String department =
      request.getParameter("department");
    int year = Integer.parseInt(
      request.getParameter("year"));

    try (Connection conn =
         DBConnection.getConnection()) {
      // 1. duplicate Student ID check
      String checkSql = "SELECT student_id "
        + "FROM students WHERE student_id = ?";
      try (PreparedStatement checkPs =
           conn.prepareStatement(checkSql)) {
        checkPs.setInt(1, studentId);
        ResultSet rs = checkPs.executeQuery();
        if (rs.next()) {
          // show "Student Already Registered!"
          response.getWriter().println(
            "<h1>Student Already "
            + "Registered!</h1> ...");
          return;
        }
      }
      // 2. insert the new student
      String sql = "INSERT INTO students "
        + "(student_id, name, email, "
        + "department, year) "
        + "VALUES (?, ?, ?, ?, ?)";
      try (PreparedStatement ps =
           conn.prepareStatement(sql)) {
        ps.setInt(1, studentId);
        ps.setString(2, name);
        ps.setString(3, email);
        ps.setString(4, department);
        ps.setInt(5, year);
        ps.executeUpdate();
      }
      response.sendRedirect("show_all");
    } catch (Exception e) {
      throw new ServletException(
        "Unable to register student.", e);
    }
  }
}
```

### 5. servlet/ShowStudentsServlet.java

```
@WebServlet("/show_all")
public class ShowStudentsServlet
    extends HttpServlet {
  protected void doGet(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    List<Student> students =
      new ArrayList<>();
    String sql = "SELECT student_id, name, "
      + "email, department, year "
      + "FROM students ORDER BY student_id";
    try (Connection conn =
           DBConnection.getConnection();
         PreparedStatement ps =
           conn.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
      while (rs.next()) {
        Student s = new Student();
        s.setStudentId(rs.getInt("student_id"));
        s.setName(rs.getString("name"));
        s.setEmail(rs.getString("email"));
        s.setDepartment(
          rs.getString("department"));
        s.setYear(rs.getInt("year"));
        students.add(s);
      }
      request.setAttribute(
        "students", students);
      request.getRequestDispatcher(
        "students.jsp")
```

```
        .forward(request, response);
    } catch (Exception e) {
      throw new ServletException(
        "Unable to display students.", e);
    }
  }
}
```

### 6. servlet/SearchStudentServlet.java

```
@WebServlet("/search")
public class SearchStudentServlet
    extends HttpServlet {
  protected void doGet(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    String search =
      request.getParameter("search");
    String sql = "SELECT * FROM students "
      + "WHERE student_id = ? "
      + "OR name LIKE ?";
    try (Connection conn =
           DBConnection.getConnection();
         PreparedStatement ps =
           conn.prepareStatement(sql)) {
      int studentId;
      try {
        studentId = Integer.parseInt(search);
      } catch (NumberFormatException e) {
        studentId = -1;
      }
      ps.setInt(1, studentId);
      ps.setString(2, "%" + search + "%");
      ResultSet rs = ps.executeQuery();
      List<Student> students =
        new ArrayList<>();
      while (rs.next()) {
        // map columns to a Student object
        // (as in ShowStudentsServlet)
      }
      request.setAttribute(
        "students", students);
      request.setAttribute(
        "searchValue", search);
      request.getRequestDispatcher(
        "search.jsp")
        .forward(request, response);
    } catch (Exception e) {
      throw new ServletException(
        "Unable to search student.", e);
    }
  }
}
```

### 7. servlet/EditStudentServlet.java

```
@WebServlet("/edit")
public class EditStudentServlet
    extends HttpServlet {
  // GET: load the record into edit.jsp
  protected void doGet(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    int studentId = Integer.parseInt(
      request.getParameter("id"));
    String sql = "SELECT * FROM students "
      + "WHERE student_id = ?";
    try (Connection conn =
           DBConnection.getConnection();
         PreparedStatement ps =
           conn.prepareStatement(sql)) {
      ps.setInt(1, studentId);
      ResultSet rs = ps.executeQuery();
      if (rs.next()) {
        Student student = new Student();
        // set the five fields from rs
        request.setAttribute(
          "student", student);
        request.getRequestDispatcher(
          "edit.jsp")
          .forward(request, response);
      } else {
        response.sendRedirect("show_all");
      }
    } catch (Exception e) {
      throw new ServletException(
        "Unable to load student.", e);
    }
  }

  // POST: save the changes
  protected void doPost(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    // read studentId, name, email,
    // department and year
    String sql = "UPDATE students SET "
      + "name=?, email=?, department=?, "
      + "year=? WHERE student_id=?";
    try (Connection conn =
           DBConnection.getConnection();
         PreparedStatement ps =
           conn.prepareStatement(sql)) {
      ps.setString(1, name);
      ps.setString(2, email);
      ps.setString(3, department);
      ps.setInt(4, year);
      ps.setInt(5, studentId);
      ps.executeUpdate();
      response.sendRedirect("show_all");
    } catch (Exception e) {
      throw new ServletException(
        "Unable to update student.", e);
    }
  }
}
```

### 8. servlet/DeleteStudentServlet.java

```
@WebServlet("/delete")
public class DeleteStudentServlet
    extends HttpServlet {
  protected void doGet(
      HttpServletRequest request,
      HttpServletResponse response)
      throws ServletException, IOException {
    int studentId = Integer.parseInt(
      request.getParameter("id"));
    String sql = "DELETE FROM students "
      + "WHERE student_id = ?";
    try (Connection conn =
           DBConnection.getConnection();
         PreparedStatement ps =
           conn.prepareStatement(sql)) {
      ps.setInt(1, studentId);
      ps.executeUpdate();
      response.sendRedirect("show_all");
    } catch (Exception e) {
      throw new ServletException(
        "Unable to delete student.", e);
    }
  }
}
```

# REFERENCES


[1] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

[2] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

[3] Eclipse Foundation. Jakarta Servlet Specification 5.0. https://jakarta.ee/specifications/servlet/

[4] Oracle. JDBC Basics – The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/

[5] Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[6] Apache Software Foundation. Apache Tomcat Documentation. https://tomcat.apache.org/

[7] Apache Software Foundation. Apache Maven Project Documentation. https://maven.apache.org/guides/

[8] OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/


# APPENDIX

**Design Checklist**

The following checklist was used to ensure a systematic approach throughout the project.

- **Problem definition and scope:** problem statement and objectives defined; required operations (CRUD, duplicate check) identified.
- **Data design:** students table with primary key and NOT NULL constraints; SQL setup script prepared.
- **Implementation:** model, utility and servlet layers created; JSP views built; parameterised queries used everywhere.
- **Validation:** each CRUD operation and the duplicate-ID check tested; results captured as screenshots.
- **Deployment and documentation:** WAR built with Maven, run on Tomcat; README, screenshots and report prepared; code published on GitHub.

# USER MANUAL

**A. Setting up the system**

1. Install JDK 21, MySQL Server, Apache Tomcat 10+ and an IDE such as NetBeans (Maven support is required).
2. Run the script `database/student_db.sql` in MySQL to create `student_db` and the `students` table.
3. Open `util/DBConnection.java` and enter your own MySQL username and password.
4. Build the Maven project and run it on Tomcat, then open `http://localhost:8080/` and the `index.jsp` page in a browser (the exact context path depends on the deployment).

**B. Using the application**

1. **Register:** enter Student ID, name, email, department and year, and click the register button. The student list opens on success (Figure 3). If the ID exists, a message is shown (Figure 4); use a different ID.
2. **View:** the student list shows all records ordered by Student ID (Figure 5).
3. **Search:** open the search page, enter a Student ID or part of a name and submit (Figure 6).
4. **Edit:** click Edit beside a student, change the name, email, department or year (the ID cannot be changed) and save (Figure 7).
5. **Delete:** click Delete beside a student to remove the record permanently (Figure 8). Take care, as there is no undo.

# GITHUB REPOSITORY

The complete project (source code, database script, README and screenshots) is published on GitHub at: **https://github.com/jindalbhumi125-dev/CampusCore-student-management**

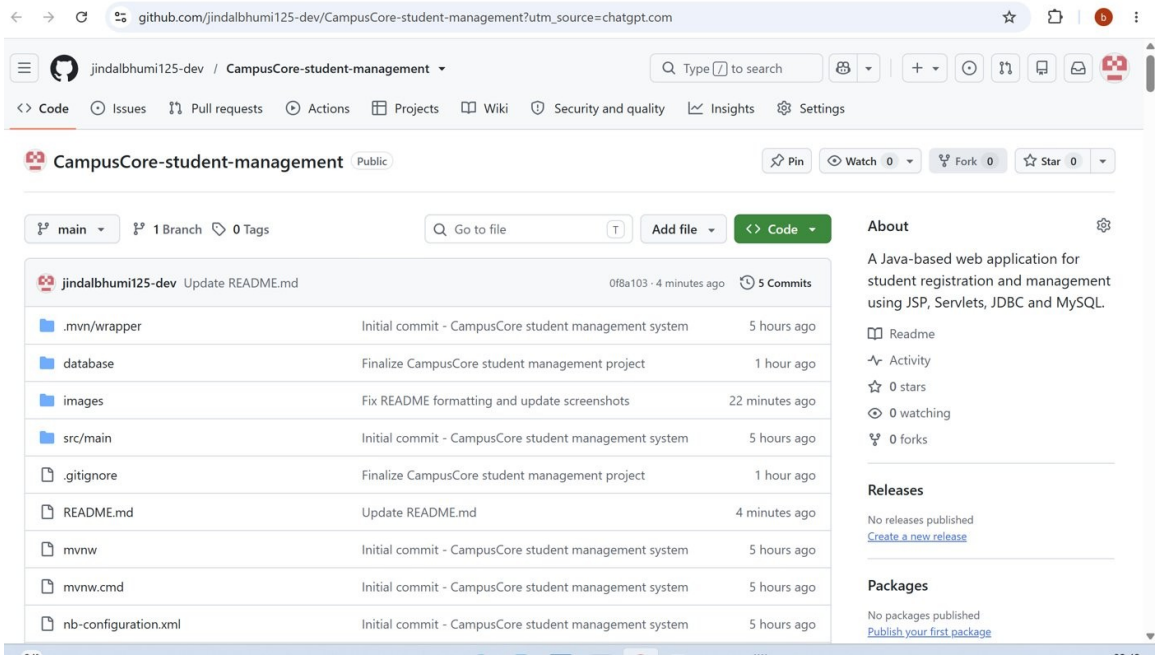



**Figure 10: GitHub repository of the project**